German International University
Faculty of Informatics and Computer Science

CNET101: Computer Networks

Spring 2026 Project

GIU-Net: UDP Chat Application

& Campus Network Simulation

Instructor
Dr. Maggie Shammaa

Teaching Assistants
Khaled Ehab
Ahmed AbdelZaher
Khadija Hossam
Mohamed Ibrahim
Mohamed Wael

Released: Tuesday, May 5, 2026 | **Deadline:** Monday, June 1, 2026 at 11:59 PM

Contents

1 Introduction

The modern Internet is built upon a carefully layered set of protocols, each solving a distinct problem in network communication. At the transport layer, TCP and UDP represent two fundamental philosophies: reliability at a cost, versus speed with no guarantees. Understanding why these mechanisms exist requires building them yourself, from scratch.

In this project, you will implement **GIU-Net** — a simple, connection-oriented chat application built directly on top of raw UDP sockets in Java. Because UDP provides no guarantees, you are responsible for implementing your own handshake, sequence numbers, acknowledgments, and basic retransmission logic. You will also observe your own packets live on the network using Wireshark.

In the second part of the project, you will move into **Cisco Packet Tracer** to design the campus network infrastructure that would realistically host and carry your GIU-Net application traffic. This gives you a complete picture of how an application you write relates to the physical and logical network it runs on.

Important Notice on Academic Integrity

This project has been deliberately designed so that AI code-generation tools (ChatGPT, Copilot, Gemini, etc.) cannot produce a working, verifiable submission. The handshake payload, port number, and network addressing scheme are all derived from your specific team's student IDs. Submissions are verified through binary session logs, Wireshark captures showing your team's actual IDs, and live evaluation challenges. Attempting to use AI will result in a **grade reduction** and a submission that is *trivially detectable*.

2 Project Objectives

By the end of this project, students will be able to:

- Understand the difference between TCP and UDP and explain why reliability mechanisms must be added manually on top of UDP.
- Implement a basic connection-oriented protocol over raw UDP using Java's `DatagramSocket` and `DatagramPacket`.
- Design and parse a simple packet structure with sequence numbers and payload data.
- Implement a custom application-level handshake, acknowledgment system, and retransmission timer.
- Capture and interpret live UDP traffic in Wireshark, identifying the application-layer payload within a captured packet.
- Apply VLSM subnetting to allocate a /22 address space efficiently across five subnets of varying sizes.
- Configure static routing between two routers in Cisco Packet Tracer and verify end-to-end connectivity.
- Configure HTTP service and a custom web page in Cisco Packet Tracer.
- Demonstrate the relationship between application-layer protocol design and network-layer infrastructure design.

3 Team Formation

- Teams consist of **exactly 4 members**. No exceptions.
- Teams must be from the same tutorial group or with the same TA.
- The student with the **lowest student ID number** in the team is automatically the **Team Leader**. The Leader's ID is used for all ID-derived calculations throughout the project.
- Team registration deadline: **Monday, May 11, 2026 at 11:59 PM** via the form that will be shared on the CMS.

4 Part 1 — GIU-Net UDP Chat Implementation

4.1 Overview

You will implement two Java programs: `GIUServer.java` and `GIUClient.java`. These programs communicate exclusively over **raw UDP sockets** (`DatagramSocket` / `DatagramPacket`). You may **not** use `Socket`, `ServerSocket`, or any TCP-based class anywhere in your submission. You may **not** use any third-party networking libraries. Standard Java I/O and `java.net` only.

Every packet sent by either side must conform to the GIU-Net packet format described below.

4.2 GIU-Net Packet Format

Every GIU-Net packet is structured as follows:

Byte(s)	Field	Size	Description
0–1	Sequence Number	2 bytes	16-bit unsigned integer, stored in **big-endian** byte order. Set to `0` for handshake and control packets. Increments by 1 for each DATA packet sent.
2+	Payload	Variable	UTF-8 encoded text. The content and meaning of the payload depends on the type of packet being sent (see Section 4.3). Empty for pure ACK packets.

Table 1: GIU-Net Packet Structure

Note: Integers are stored in **big-endian (network) byte order**. Be careful when writing and reading the 2-byte sequence number field — Java's `byte` type is signed, so use `Byte.toUnsignedInt()` when reading individual bytes.

4.3 Packet Types

The type of a packet is determined entirely by the content of its Payload field. The following payload strings have special meaning:

Payload Content	Direction	Meaning
`HELLO-GIU-<LeaderID>-<Sum>`	Client → Server	Handshake initiation (see Section 4.4)
`OK-GIU-<LeaderID>-<Sum>`	Server → Client	Handshake accepted
`REJECTED`	Server → Client	Handshake refused (wrong team string)
`ACK:<seqnum>`	Either direction	Acknowledgment of a DATA packet with that sequence number
`EXIT`	Either direction	Request to close the connection
Any other text	Either direction	A chat message (DATA packet)

Table 2: GIU-Net Payload Types

4.4 The Handshake

Before any chat messages can be exchanged, the client and server must complete a 2-step handshake. The socket on the client side must **not** be opened until the user types the word `CONNECT`.

1. **Client → Server:** A packet with Seq = 0 and the following payload:

```
"HELLO-GIU-" + LeaderID + "-" + SumOfLastDigits
```

2. **Server → Client:**
   - If the received string matches the expected value (computed from the same formula using the server team's IDs): reply with payload `"OK-GIU-<LeaderID>-<Sum>"` and Seq = 0. Connection is now established.
   - If the string does not match: reply with payload `"REJECTED"` and close the socket.

Team Verification String: The `<Sum>` in the handshake payload is the arithmetic sum of the **last digit** of each team member's student ID.

Worked Example

Team IDs: `20012345`, `20019876`, `20014321`, `20016780`
Last digits: 5, 6, 1, 0 ⇒ Sum = 12

Client sends payload: `"HELLO-GIU-20012345-12"`
Server replies with: `"OK-GIU-20012345-12"`

The server must validate this string **exactly**. Any mismatch — including wrong leader ID, wrong sum, or wrong format — must cause the server to send `"REJECTED"` and close the connection.

4.5 Required Protocol Behaviours

After the handshake completes successfully, both programs must support the following behaviours.

4.5.1 Bi-Directional Chat

- Both the server and the client continuously read lines of text from the keyboard (`stdin`).
- Each line typed is sent as a DATA packet with an incrementing sequence number (starting from 1).
- When a DATA packet is received, the program prints the message to `stdout` with the prefix `[RECEIVED]` and immediately sends back an acknowledgment packet with payload `"ACK:<seqnum>"`.
- Both programs run their **sending** and **receiving** logic in **separate threads** so that typing and receiving happen simultaneously without blocking each other.

4.5.2 Acknowledgment and Retransmission

- After sending a DATA packet, the sender waits up to **3 seconds** for the corresponding `ACK:<seqnum>` to arrive.
- If the ACK does not arrive within 3 seconds, the sender prints `[TIMEOUT] Retrying...` and retransmits the packet **once**.
- If the ACK still does not arrive after the retransmission, the sender prints `[CONNECTION LOST]` and both programs must exit cleanly.

4.5.3 Connection Teardown

- Either side can close the connection by typing `EXIT`.
- The program sends a packet with payload `"EXIT"` to the other side.
- Upon receiving `"EXIT"`, the other side prints `[Connection closed by remote.]` and both programs close their sockets and terminate.

4.6 Port Number

- The server listens on a port derived as follows: take the **last 4 digits** of the Leader's ID. If the result is below 1024, add 1000.
- **Example:** Leader ID `20012345` ⇒ port `2345`. Leader ID `20010800` ⇒ port `0800` = 800, add 1000 ⇒ port `1800`.

- Both `GIUServer.java` and `GIUClient.java` must hardcode this port. Write the derivation formula as a comment at the top of both files.

4.7 Provided `.class` File

You are provided with one pre-compiled Java class file. Its source code is **not available**. You must call it exactly as documented. This file is compiled for **Java 17+**.

4.7.1 `GIULogger.class`

```
// Call this AFTER every send AND after every receive.
// direction: either "SEND" or "RECV"
// packetBytes: the raw byte array of the full packet sent or received
// timestamp: use System.currentTimeMillis()
// Writes a binary log entry to "giunet_session.log" in the working
   directory.
void GIULogger.log(String direction, byte[] packetBytes, long timestamp
   )
```

The `giunet_session.log` file is a **required deliverable**. It is a binary file. It automatically embeds your machine's hostname and a verified timestamp in each entry. **A session log that was not produced by actually running the code on a real machine is trivially detectable.**

The class file will be distributed via the CMS. Place it in the same directory as your `.java` files and compile everything together.

4.8 Network Setup & Wireshark Capture

- The client and the server must run on **two separate physical machines** connected via a WLAN or mobile hotspot. Running both on the same machine using `localhost` is **not accepted**.
- To find the server machine's IP address, run `ipconfig` (Windows) or `ifconfig` (Linux/Mac) on the server machine and note the IPv4 address. Hardcode this address into `GIUClient.java`.
- While the chat session is running, capture the traffic on **either machine** using Wireshark.

Wireshark filter to use:

```
udp.port == <your derived port number>
```

Your Wireshark capture must clearly show:

1. The **HELLO** handshake packet from the client, with the full team verification string visible in the packet's data field.
2. The **OK** reply from the server.
3. At least **3 DATA packets** containing chat messages.

4. At least **3 ACK packets** in response.

Because the payload is plain UTF-8 text, Wireshark will display it as readable characters directly in the *Data* field — no hex conversion needed. You must annotate the submitted screenshot to label the HELLO, OK, DATA, and ACK packets.

5 Part 2: Campus Network Simulation (Cisco Packet Tracer)

5.1 Overview

You will design and simulate the GIU campus network in Cisco Packet Tracer. This network is the infrastructure that would realistically carry your GIU-Net traffic between buildings. You will apply subnetting, configure routers, enable HTTP services, and optionally configure security access rules.

5.2 Network Topology

You must design a network with the following components:

- **Switch A** — connected to 4 PCs representing the Student Lab (Subnet B)
- **Switch B** — connected to 4 PCs representing Student Housing (Subnet C)
- **Switch C** — connected to a Web Server and the GIU-Net Server device (Subnet E — DMZ)
- **Switch D** — connected to 2 Staff PCs (Subnet A)
- **Router 1** — connects Switch A, Switch D, Switch C, and the WAN link
- **Router 2** — connects Switch B and the WAN link
- The WAN link between Router 1 and Router 2 is Subnet D

5.3 IP Addressing — VLSM from Leader's ID

The base network is derived from the **Team Leader's student ID** as follows:

1. Take the 8-digit ID: e.g., `20012345`
2. Form the IP using digit pairs: digits 1–2 . digits 3–4 . digits 5–6 . `0` ⇒ `200.12.34.0`
3. Apply prefix length /**22**
4. Base network: `200.12.34.0/22` (covers `200.12.32.0` – `200.12.35.255`, 1022 usable hosts)

Note: If any octet exceeds 255, subtract 100 from it until it falls within range.

Apply **VLSM** (Variable Length Subnet Masking) to carve the base /22 into the following five subnets. You must allocate them in **order from largest to smallest** to minimise address space waste:

Subnet	Purpose	Required Hosts	Connects To
C	Student Housing	200	Switch B
B	Student Lab	100	Switch A
A	Staff Offices	50	Switch D
E	Server DMZ	10	Switch C
D	Router WAN Link	2	Router 1 ↔ Router 2

Table 3: Subnet Allocation Requirements

5.4 Required Tasks

5.4.1 Task 1 — VLSM Subnet Table

Produce a complete subnet table with the following columns for each of the five subnets: Subnet label, Network Address, Subnet Mask (dotted decimal), Prefix Length, First Usable IP, Last Usable IP, Broadcast Address, and which device is assigned which IP. Show your calculations manually in the report — do not just paste the output of an online calculator.

5.4.2 Task 2 — Static Routing Configuration

Configure static routes on Router 1 and Router 2 such that every subnet can reach every other subnet. Verify by successfully pinging from a Housing PC (Subnet C) to the GIU-Net Server (Subnet E). Take a screenshot of the successful ping in **Simulation Mode** showing the packet path hop by hop.

5.4.3 Task 3 — HTTP Service & Custom Team Webpage

1. Enable the HTTP server on the **Web Server** device in Subnet E.
2. Edit the `index.html` file on the Web Server to display your **team name** and the **full name and student ID of every team member**. The page must be clearly readable when loaded in Packet Tracer's browser.
3. From a PC in Subnet B, open Packet Tracer's built-in web browser and navigate to the Web Server's IP address to load your custom page.
4. Screenshot required showing the browser with your team's page fully loaded.

5.4.4 Task 4 — Simulation Mode Packet Trace

Using Packet Tracer's Simulation Mode, trace a single ICMP echo request from `Lab-PC-1` (Subnet B) to the GIU-Net Server (Subnet E). Capture screenshots showing the packet at **every hop**, including the encapsulation/decapsulation at each router interface. Annotate each screenshot to identify the source IP, destination IP, and which layer (Network / Data Link) is responsible for the processing shown.

5.4.5 Task 5 — Access Control List (Bonus)

Configure an ACL on Router 2 that enforces the following policy:

- **Deny** all traffic originating from Subnet C (Student Housing) destined for Subnet E (DMZ/Server).
- **Permit** all other traffic.

Demonstrate the ACL is working by showing:

1. A Housing PC **cannot** ping the GIU-Net Server (ping fails).
2. A Lab PC **can** ping the GIU-Net Server (ping succeeds).

In your documentation, write a short paragraph explaining how this ACL relates to the HELLO/REJECTED handshake logic in Part 1 — both enforce access control, but at different layers of the network stack.

6 Deliverables

Submit a single **ZIP file** named `GIUNet_TeamLeaderID_SS26.zip` containing the following:

1. `GIUServer.java` — Fully commented source code.
2. `GIUClient.java` — Fully commented source code.
3. `giunet_session.log` — Binary session log produced by an actual live run between **two physical machines** (not localhost). The log must show at minimum: the HELLO/OK handshake and at least 5 chat messages exchanged.
4. **Wireshark capture (`.pcapng`)** — Captured during the live session, filtered to your server's UDP port. Must show the HELLO handshake, OK reply, at least 3 DATA packets, and at least 3 ACK packets, as described in Section 4.7.
5. **Cisco Packet Tracer file (`.pkt`)** — Complete and working network topology.
6. **Project Report (PDF)** — See Section 6.1.

6.1 Project Report Requirements

The report must include:

1. **Team information:** Full names, student IDs, derived port number (with the derivation shown step by step), and the team's HELLO/OK handshake string (with the sum derivation shown).
2. **Protocol design choices:** Explain why you chose your retransmission timeout value and how you handled the case where a thread is waiting for an ACK while the other thread is trying to read from the keyboard. These answers must be specific to your implementation.

3. **A real debugging story:** Describe *one specific bug* you encountered during development. Include what the symptom was, what you initially thought the cause was, how you diagnosed it (e.g., using Wireshark or print statements), and what the actual fix was. This section will be discussed during evaluation.
4. **VLSM subnet table** for all five subnets with full manual calculations shown.
5. **Routing tables** for Router 1 and Router 2 as configured in Packet Tracer.
6. **All required screenshots** from Part 2 Tasks 2–4, with annotations as described.
7. **(Bonus)** If Task 5 is completed: include the ACL configuration commands and the explanation paragraph.

7 Evaluation

Evaluation will take place during the week of **June 2–8, 2026**. A Doodle scheduling sheet will be sent via CMS to reserve your team's evaluation slot. **All team members must be present.**

During evaluation, each team will be given a **Live Modification Challenge** — a small, unseen change to the protocol (for example: change the handshake keyword from `HELLO` to a different word, or change the timeout from 3 seconds to 2 seconds). The team must implement and demonstrate the change **on the spot within the evaluation slot**. This is only possible if the team genuinely understands their own code.

Additionally, the evaluator can point to any part of your source code and ask any team member to explain what it does and what would happen if it were removed or changed. **Answers should be related to the course concepts, and vague answers are not accepted**.

8 Grading Criteria

Component	Sub-criteria	Weight
Part 1 — UDP Chat Implementation	Correct packet structure & handshake	10%
	Bi-directional chat working correctly	10%
	Retransmission & ACK logic	10%
Part 2 — Packet Tracer	VLSM subnetting correctness	10%
	Routing & connectivity verified	10%
	HTTP service & custom team webpage	10%
	ACL configuration **(Bonus)**	5%
Deliverables	Session log & Wireshark capture	10%
	Report quality & debugging story	10%
Evaluation	Individual code comprehension Q&A	20%
	Total	**100% + 5% Bonus**

Table 4: Project Grading Rubric

9 Deadlines

Milestone	Deadline
Team Registration	Monday, May 11, 2026 at 11:59 PM
Final Submission (ZIP on CMS)	Monday, June 1, 2026 at 11:59 PM
Evaluation Week	June 2–8, 2026 at designated slots

Table 5: Project Deadlines

Late Submission Policy

Submissions received after the deadline will be penalised at **10% per day**. Submissions more than 5 days late will receive 50% penalty and no reservation for evaluation. No extensions will be granted except in documented emergencies submitted to Dr. Maggie Shammaa directly.

10 Useful Resources

- Java `DatagramSocket` documentation: https://docs.oracle.com/en/java/docs/api/java.base/java/net/DatagramSocket.html

- Wireshark download: https://www.wireshark.org
- Cisco Packet Tracer (available via Cisco NetAcad): https://www.netacad.com
- Provided `GIULogger.class` file: distributed via CMS
- VLSM calculator (for checking your work only — show manual calculations in report): https://www.subnet-calculator.com/vlsm.php

Best of luck to all teams!

Remember: the goal is not just to submit working code. The goal is to understand networking deeply enough that you could explain every packet you send.